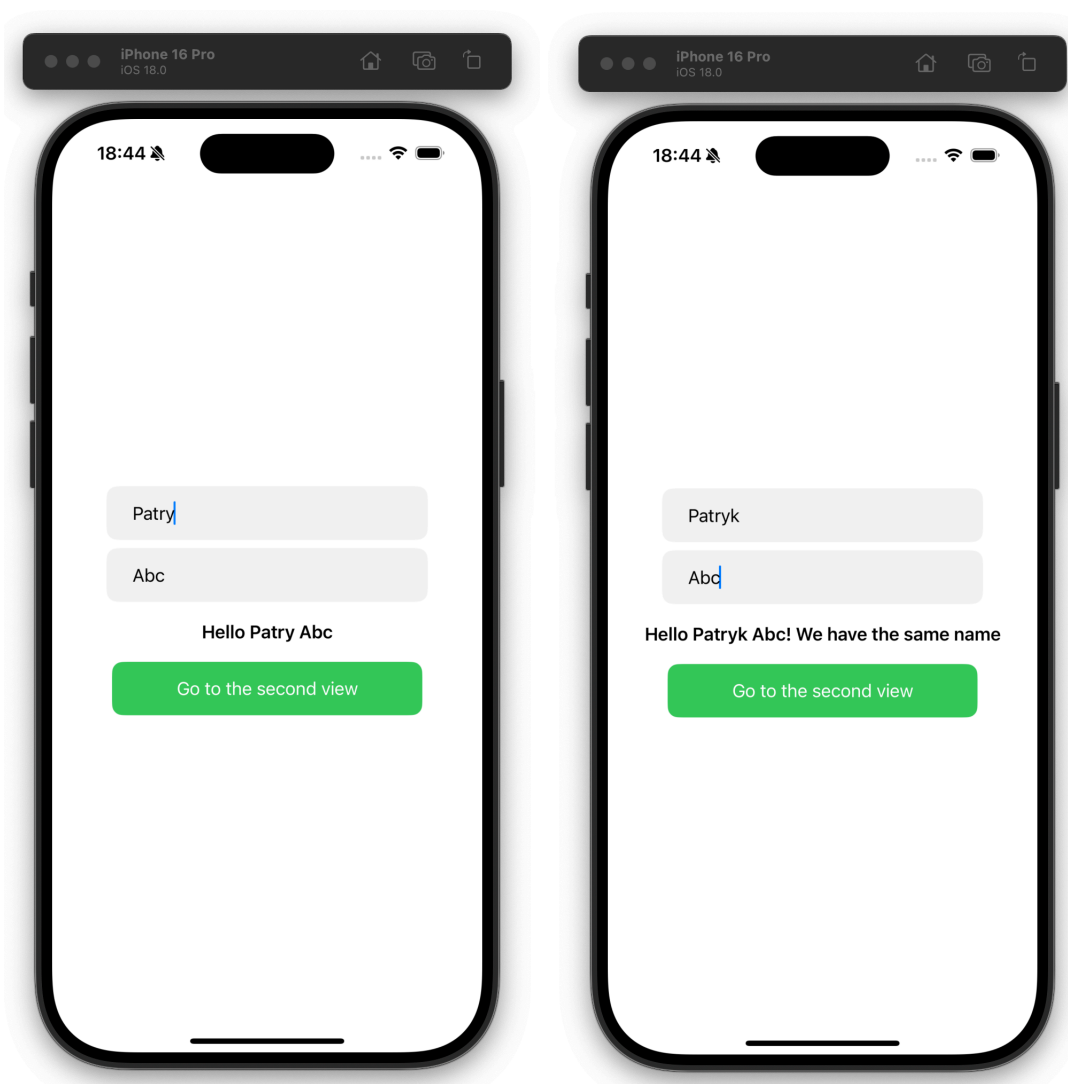


Zadanie 1 – Wprowadzenie do iOS i Swift

Należy stworzyć widok spełniający poniższe wymagania:

1. Można podać imię i nazwisko.
2. Za każdym razem, gdy użytkownik wpisze dowolną literę w polu imienia lub nazwiska, należy zaktualizować widoczny tekst, tak aby zawierał „Hello imię nazwisko” lub „Witaj imię nazwisko”.
3. Dla wybranego imienia należy dodać komunikat „We have the same name” lub „Mamy to samo imię”.
4. Stworzyć przycisk „Go to the second view” (lub „Idź do drugiego widoku”), po którego kliknięciu otworzy się drugi widok.
5. Drugi widok powinien zawierać dwie kontrolki: tekst „Your surname” (lub „Twoje nazwisko”) oraz pole tekstowe z przekazaną wartością nazwiska z pierwszego widoku. Po zmianie nazwiska i powrocie do pierwszego widoku zmiana powinna być widoczna.
6. Wszystkie kontrolki powinny mieć między sobą odstępy (padding), np. 10 pikseli. Tło przycisku powinno być ustawione na dowolny kolor inny niż domyślny.



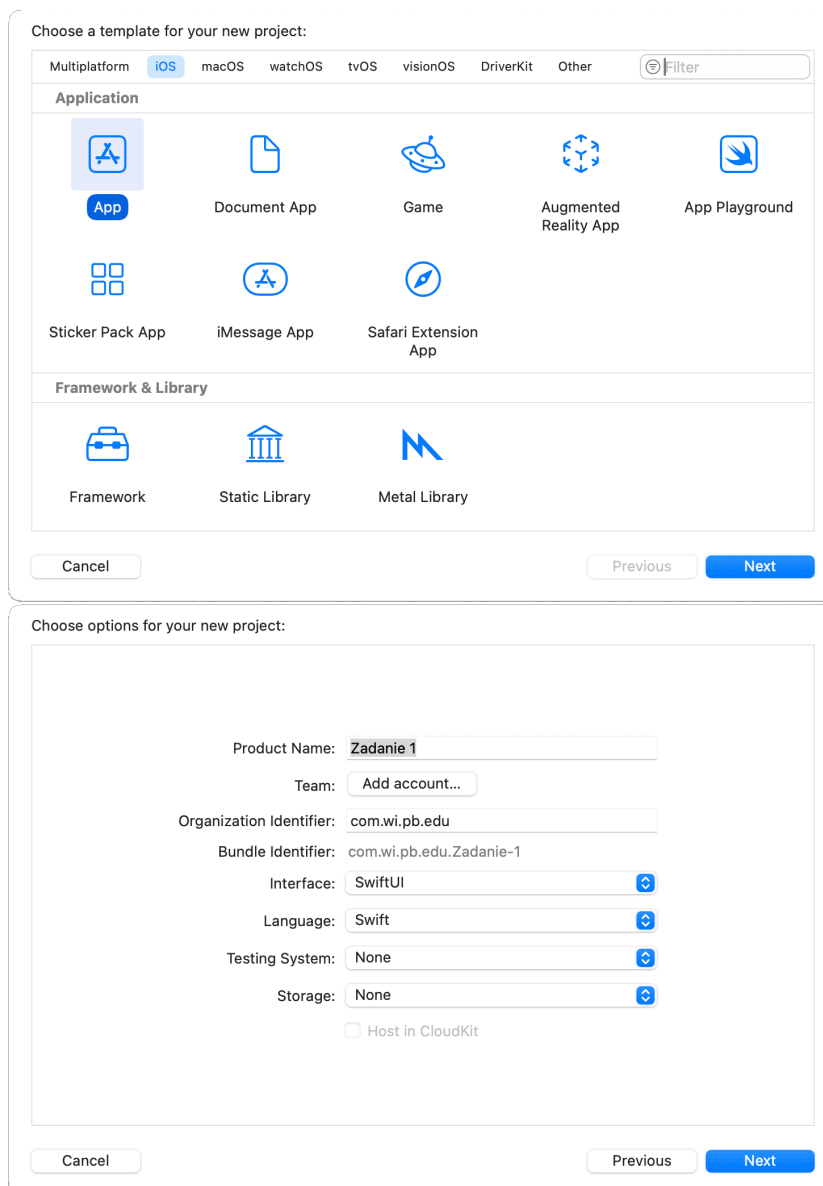
Przykładowy wygląd pierwszego widoku z reakcją na podanie wybranego imienia.

Instrukcja do zadania 1

1. Do tworzenia aplikacji w języku **Swift** używamy środowiska **Xcode**:



2. Po uruchomieniu programu wybieramy standardowy szablon **App** i tworzymy nowy projekt w technologii **Swift/SwiftUI**:

The image shows two screenshots of the Xcode project creation interface. The top screenshot, titled 'Choose a template for your new project:', shows the 'Application' section with the 'App' template selected. The bottom screenshot, titled 'Choose options for your new project:', shows the configuration options for the new project. The 'Product Name' is 'Zadanie 1', 'Team' is 'Add account...', 'Organization Identifier' is 'com.wi.pb.edu', 'Bundle Identifier' is 'com.wi.pb.edu.Zadanie-1', 'Interface' is 'SwiftUI', 'Language' is 'Swift', 'Testing System' is 'None', and 'Storage' is 'None'. There is also an unchecked checkbox for 'Host in CloudKit'.

- **Product name** to nazwa projektu.
- **Interface** to framework, którego chcemy używać. Na większości urządzeń dostępne będą opcje **SwiftUI** oraz **Storyboard (UIKit)**. UIKit to starszy i bardziej złożony framework, dlatego do nowych projektów zalecany jest **SwiftUI**. Opcja SwiftUI jest widoczna gdy językiem projektu jest Swift.
- **Language** to język, w którym będzie tworzona aplikacja. Najczęściej dostępne są **Objective-C** oraz **Swift**. Swift jest językiem nowszym, prostszym i zalecanym. Na komputerach politechniki dostępny jest SwiftUI i Storyboard (UIKit) przy użyciu Swifta; Objective-C obsługuje wyłącznie Storyboard.

3. Po utworzeniu projektu pojawi się plik zawierający strukturę `struct ContentView: View`.

4. Dodajmy do aplikacji trzy podstawowe kontrolki: dwa pola tekstowe i etykietę (tekst tylko do odczytu). Użyjemy struktur `TextField` oraz `Text`. Umieszczamy je wewnątrz widoku `VStack`, który układa elementy pionowo (Vertical Stack), a całość opakowujemy w `NavigationView`, co umożliwi nawigację do drugiego widoku. Struktura `ContentView` będzie wyglądała następująco:

```
1 import SwiftUI
2
3 struct ContentView: View {
4     var body: some View {
5         NavigationStack {
6             VStack {
7                 TextField("Enter your name")
8                 TextField("Enter your surname")
9                 Text("Some text")
10            }
11        }
12    }
13 }
14
15 #Preview {
16     ContentView()
17 }
```

Zwróć uwagę na ostatnie linie (15–17) pliku, które umożliwiają podgląd aplikacji. Ich usunięcie nie spowoduje błędu, ale wyłączy podgląd widoku (zazwyczaj widoczny po prawej stronie).

5. Do obsługi imienia i nazwiska dodajemy dwie prywatne zmienne z atrybutem `@State`. Atrybut ten pozwala zarządzać mutowalnym stanem w aplikacji. Deklaracja zmiennej ma postać: `@State private var varName: typeName = defaultValue`. Dla imienia i nazwiska, które są tekstem, zapis będzie wyglądał tak:

```
1 @State private var firstName: String = ""
2 @State private var surname: String = ""
```

6. Aby powiązać nowe zmienne z widokiem, należy zmodyfikować konstruktory `TextField`, dodając jako drugi (nazwany) parametr `text` odpowiednią zmienną – `firstName` lub `surname`. Składnia wygląda tak: `text: $firstName`. Dzięki temu pole zostanie powiązane ze stanem.

7. Wygląd kontrolki można modyfikować za pomocą łańcucha metod wywoływanych po konstrukcji danej struktury. Przykładowo, dodanie odstępu:

```
1 TextField("Enter your name", text: $firstName)
2     .padding(10)
```

Dostępne są również inne modyfikatory, np. `background`, `frame`, `cornerRadius`, `textInputAutocapitalization`, `font`, `foregroundColor`. Metoda `frame` pozwala ustawić rozmiar kontrolki, przyjmując parametry `width` i `height`.

8. Dodaj kolejną prywatną zmienną z atrybutem `@State` o nazwie `message`, która będzie powiązana z kontrolką `Text`. Po jej dodaniu należy zmienić konstrukcję etykiety z `Text("Some text")` na `Text(message)`. Można również dodać modyfikatory, np. `padding`.

9. Zmienna `message` powinna aktualizować się w zależności od wartości `firstName` i `surname`. W tym celu do `TextField` należy dodać metodę `onChange`, np.:

```

1 .onChange(of: firstName) { oldValue, newValue in
2     message = newValue
3 }

```

Na tym etapie aplikacja powinna wyświetlać wpisywane imię w etykiecie **Text**. Warto sprawdzić, czy projekt się poprawnie kompiluje.

10. Na początku struktury **ContentView** dodaj stałą, która będzie przechowywać imię powodujące wyświetlenie dodatkowego komunikatu w **Text**. Użyj słowa kluczowego **let** i nadaj jej nazwę **myName**. Nie wymaga ona atrybutu **@State**, ponieważ nie jest powiązana z widokiem.

11. Utwórz metodę, która będzie aktualizować wartość **message**:

```

1 func updateMessage(newFirstName: String, newSurname: String) -> String {
2     return "\(newFirstName) \(newSurname)"
3 }

```

Podłącz ją do metody **onChange** zamiast bezpośredniego przypisania:

```

1 .onChange(of: firstName) { oldValue, newValue in
2     message = updateMessage(newFirstName: newValue, newSurname: surname)
3 }

```

Aby uwzględnić zmianę nazwiska, należy dodać analogiczną metodę **onChange** do pola **surname**. W ten sposób **message** będzie aktualizowane przy każdej zmianie imienia lub nazwiska.

12. Zaktualizuj metodę **updateMessage**, aby uwzględniała wartość **myName**. Jeśli **newFirstName == myName**, funkcja powinna zwracać: **"Hello \(newFirstName) \(newSurname)! We have the same name"** w przeciwnym wypadku: **"Hello \(newFirstName) \(newSurname)"**.

13. Ostatnim krokiem jest utworzenie drugiego widoku. Dodaj w nim kontrolki **Text** i **TextField**, a następnie połącz go z pierwszym widokiem. Najprościej zrobić to przy pomocy struktury **NavigationLink**, np.:

```

1 NavigationLink(destination: SecondView(surname: $surname)) {
2     Text("Go to the second view")
3         .padding(10)
4         .cornerRadius(10)
5 }

```

14. W strukturze **SecondView** zmienna **surname** powinna być oznaczona atrybutem **@Binding** i nie powinna być prywatna. Domyślny modyfikator dostępu (**internal**) jest wystarczający.

15. W **SecondView** należy dostosować sekcję **Preview**, ponieważ dodanie parametru **surname** wymaga jego przekazania w konstruktorze. Można to zrobić następująco:

```

1 #Preview {
2     SecondView(surname: .constant("Surname"))
3 }

```

16. Należy dokończyć i przetestować aplikację w poszukiwaniu błędów.